Exp - 4 Machine Learning for IoT Applications

Linear Regression, Decision Tree & Random Forest for Sensor Data Regression and Classification

1. Aim

To implement three beginner-friendly Supervised Learning algorithms — Linear Regression, Decision Tree, and Random Forest — on IoT sensor data, using a simulated smart-room dataset, without requiring any physical hardware.

2. Objectives

- Understand the difference between supervised regression and supervised classification.
- Simulate a realistic multi-sensor IoT dataset (temperature, humidity, light, CO2) entirely in software.
- Train and evaluate a Linear Regression model to predict energy consumption.
- Train and evaluate a Decision Tree classifier to predict room occupancy, and compare it against a Random Forest ensemble.
- Interpret model outputs (accuracy, R^2, MAE, confusion matrix) in the context of an IoT application.

3. Software Requirements

Python 3.8+, Google Colab (recommended) or Jupyter/VS Code, and the libraries numpy, pandas, matplotlib, scikit-learn (pre-installed on Colab).

4. Key Concepts (Quick Reference)

Feature (X): a sensor input — temperature_C, humidity_pct, light_lux, co2_ppm. Label / Target (y): the value to predict — energy_kWh (number) or occupancy (category). Every model follows the same recipe: (1) split data into train/test, (2) .fit() to train, (3) .predict() on unseen data, (4) compare predictions to the real values.

Algorithm	Learns from	Predicts	Used here for
Linear Regression	Input → known number	Continuous value	Energy consumption (kWh)
Decision Tree	Input → known class	Discrete category	Room: Occupied / Empty
Random Forest	Input → known class (ensemble)	Discrete category	Room: Occupied / Empty

5. The Simulated IoT Dataset

A “Smart Room” is simulated with 4 sensors — temperature_C, humidity_pct, light_lux, co2_ppm — plus two derived targets: occupancy (0/1, classification target) and energy_kWh (regression target). The code below generates 500 rows of realistic, noisy sensor readings and saves them to a CSV; every step after this simply reads that CSV, exactly as it would read data logged from real sensors.

```
import numpy as np, pandas as pd
np.random.seed(42)
n = 500
temperature = np.random.normal(25, 4, n).clip(15, 38)
humidity = np.random.normal(50, 10, n).clip(20, 90)
light = np.random.normal(300, 200, n).clip(0, 900)
co2 = np.random.normal(600, 250, n).clip(350, 2000)

# occupancy depends on light + CO2 (+ noise); energy depends on
# temperature + light + occupancy (+ noise) — see full script for exact formulas
occupancy = ...   # 0 = Empty, 1 = Occupied
energy = ...      # energy_kWh

df = pd.DataFrame({"temperature_C": temperature.round(2),
    "humidity_pct": humidity.round(2), "light_lux": light.round(1),
    "co2_ppm": co2.round(1), "occupancy": occupancy,
    "energy_kWh": energy.round(2)})
df.to_csv("smart_room_iot_dataset.csv", index=False)
```

Part A — Supervised Learning Algorithms

A.1 Linear Regression — Predicting Energy Consumption

Goal: predict the numeric energy consumption (kWh) from sensor readings — a regression problem. Linear Regression fits energy as a weighted sum of the inputs (energy = w1·temp + w2·humidity + … + c), finding the weights that minimise total squared error between predicted and actual energy across all training rows ("Least Squares").

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score, mean_absolute_error

df = pd.read_csv("smart_room_iot_dataset.csv")
X = df[["temperature_C","humidity_pct","light_lux","co2_ppm","occupancy"]]
y = df["energy_kWh"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

model = LinearRegression()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

print("R2 score:", round(r2_score(y_test, y_pred), 3))
print("MAE:", round(mean_absolute_error(y_test, y_pred), 3))
```

```
R2 score: 0.909
MAE: 0.599
Learned weights: {'temperature_C': 0.154, 'humidity_pct': 0.002,
'light_lux': 0.01, 'co2_ppm': 0.0, 'occupancy': 1.614}
```

R^2 = 0.91 means the model explains ~91% of the variation in energy consumption. MAE = 0.6 means predictions are off by about 0.6 kWh on average. Occupancy has by far the largest weight — an occupied room uses noticeably more energy — followed by temperature; humidity and CO2 barely matter.

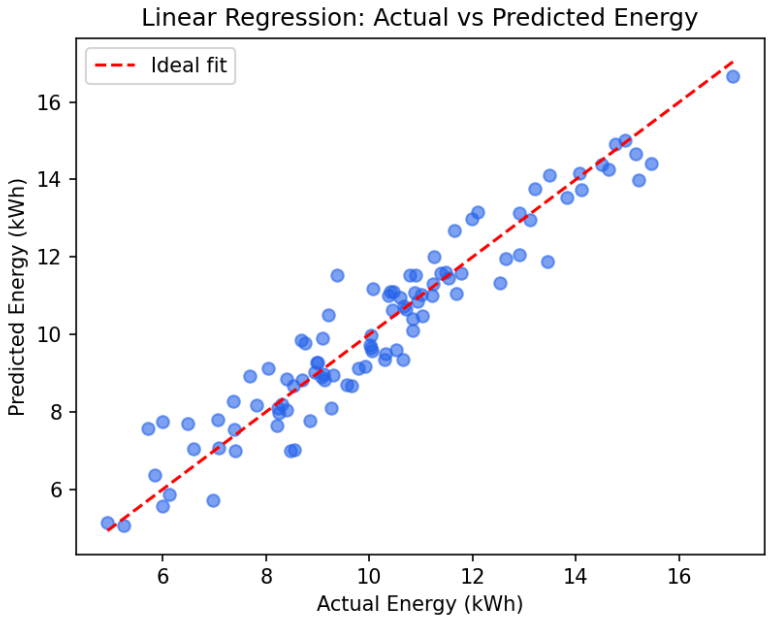


Fig. A.1 — Predicted vs Actual energy consumption. Points closer to the red dashed line are better predictions.

A.2 Decision Tree — Classifying Room Occupancy

Goal: predict whether the room is Occupied (1) or Empty (0) from sensor readings — a classification problem. A Decision Tree learns a sequence of yes/no questions on the features, picking at each node the split that produces the purest possible groups (lowest Gini impurity). The tree is limited to max_depth = 3 to avoid overfitting and to keep it readable.

```
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import accuracy_score, confusion_matrix

X = df[["temperature_C","humidity_pct","light_lux","co2_ppm"]]
y = df["occupancy"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)

clf = DecisionTreeClassifier(max_depth=3, random_state=42)
clf.fit(X_train, y_train)
y_pred = clf.predict(X_test)
print("Accuracy:", round(accuracy_score(y_test, y_pred), 3))
```

```
Accuracy: 0.7
```

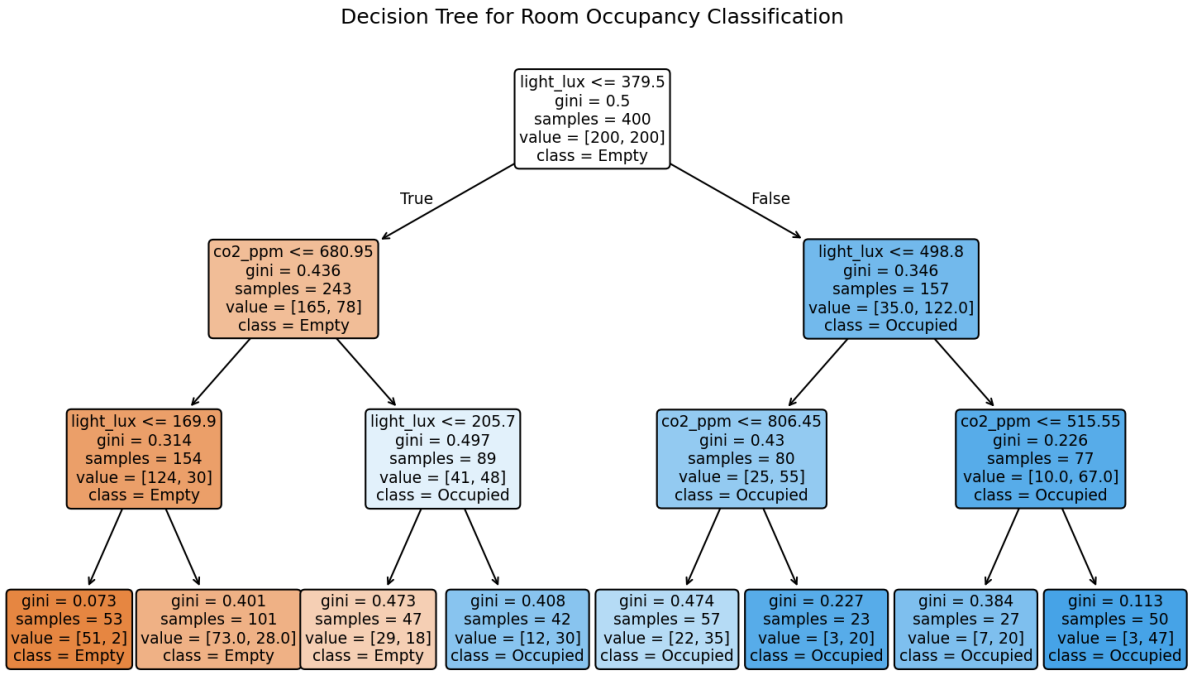


Fig. A.2a — The learned decision tree (depth limited to 3 for readability).

A confusion matrix shows what kind of mistakes the model makes: TN = 37, FP = 13, FN = 17, TP = 33 (Accuracy = (33+37)/100 = 70%). A False Negative here means the system thinks the room is empty when it isn't (lights switched off on an occupied room); a False Positive keeps the AC running in an empty room.

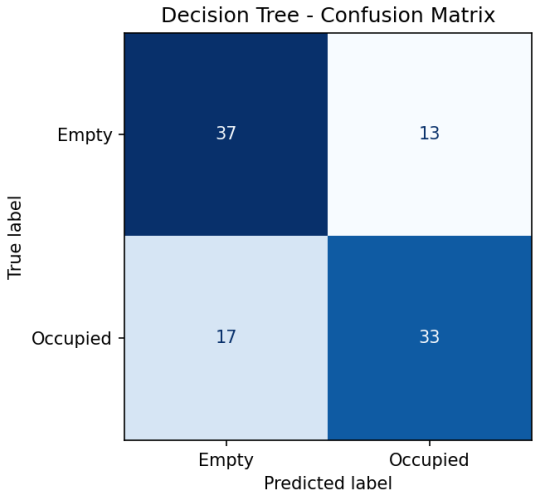


Fig. A.2b — Confusion matrix: rows are actual class, columns are predicted class.

Part B — Random Forest (Ensemble Extension)

A natural next step after a single Decision Tree is a Random Forest: instead of one tree, it builds many trees (an “ensemble”), each trained on a random subset of the data, and lets them vote on the final prediction. This usually improves accuracy and reduces overfitting compared to a single tree, at the cost of losing the easy readability of one tree diagram.

B.1 Code — Same Pipeline, Different Classifier

Only the classifier changes: RandomForestClassifier (from sklearn.ensemble) replaces DecisionTreeClassifier. Features, train/test split, and evaluation stay identical.

```
from sklearn.ensemble import RandomForestClassifier   # ← changed

X = df[["temperature_C","humidity_pct","light_lux","co2_ppm"]]
y = df["occupancy"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)

clf = RandomForestClassifier(random_state=42)
clf.fit(X_train, y_train)
y_pred = clf.predict(X_test)
print("Accuracy:", round(accuracy_score(y_test, y_pred), 3))
```

```
Accuracy: 0.66
```



```
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, ConfusionMatrixDisplay

X = df[["temperature_C", "humidity_pct", "light_lux", "co2_ppm"]]
y = df["occupancy"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)

clf = RandomForestClassifier(random_state=42)
clf.fit(X_train, y_train)
y_pred = clf.predict(X_test)

print("Accuracy:", round(accuracy_score(y_test, y_pred), 3))
```

```
Accuracy: 0.66
```

Fig. B.1 — Random Forest code and printed accuracy output (Google Colab).

The confusion matrix below shows exactly where the Random Forest's mistakes differ from the Decision Tree's: TN = 33, FP = 17, FN = 17, TP = 33. Compared to the Decision Tree (TN = 37, FP = 13, FN = 17, TP = 33), the Random Forest kept the same number of missed detections (FN = 17) but raised false alarms from 13 to 17 — it more often predicted "Occupied" when the room was actually empty.

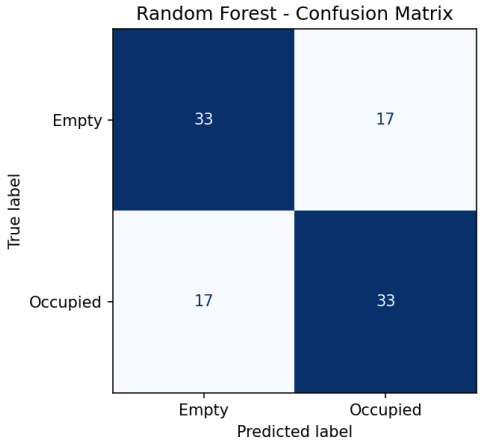


Fig. B.2 — Random Forest confusion matrix: rows are actual class, columns are predicted class.

B.2 Comparison & Interpretation

Algorithm	Target predicted	Result
Decision Tree (max_depth=3)	occupancy (0/1)	Accuracy = 70%
Random Forest (default)	occupancy (0/1)	Accuracy = 66%

The single Decision Tree (0.70) slightly outperformed the Random Forest (0.66) here. This is not unusual: Random Forest's advantage — reducing variance by averaging many trees — shows the biggest benefit on larger or noisier datasets. With only 500 rows and a fairly clean simulated relationship, a single well-tuned shallow tree already generalises well, so the forest's added randomness has less noise to average out — while losing the single tree's key advantage of being directly readable as if/else rules.

> *Note: Random Forest performance can often be improved by tuning n_estimators or max_depth. Defaults were used here to keep the comparison with the Decision Tree direct.*

6. Summary of Results

Algorithm	Type	Target predicted	Result
Linear Regression	Supervised — Regression	energy_kWh (numeric)	R2 = 0.91, MAE = 0.60
Decision Tree	Supervised — Classification	occupancy (0/1)	Accuracy = 70%
Random Forest	Supervised — Classification (ensemble)	occupancy (0/1)	Accuracy = 66%

7. Conclusion

A simulated multi-sensor IoT dataset was created entirely in software, replacing the need for physical hardware. Three supervised algorithms were applied to it: Linear Regression to predict a numeric energy value, Decision Tree to predict room occupancy, and Random Forest as an ensemble extension of the tree classifier. The same code structure (load data → split → fit → evaluate → visualize) applies directly to real sensor data collected from Arduino, Raspberry Pi, or any IoT gateway, making this a realistic first step before working with physical hardware. Comparing the Decision Tree and Random Forest also illustrated an important ML lesson: a more complex ensemble model is not automatically better — its benefit depends on the size and noise level of the dataset.